MarinSpin: Can a Transformer Learn Coarsening Dynamics?

Reproducing 2D-Ising kinetic Monte-Carlo *rates and waiting times* from trajectory data alone

Y. Elmatad R. Wojdyla D. Hall **Open Athena** Built on **Marin** | marin.community yael.elmatad@openathena.ai

code & data

Takeaways

Learns kinetic Monte Carlo (KMC) from data. A vanilla transformer recovers flip *rates* and *waiting times* from trajectories alone.

Grammar matters. MarinSpin is off-the-shelf; much of the modeling work lives in the *tokenization & grammar*.

Matches the true dynamics. Per-step KL near the oracle floor; reproduces $\xi(T)$ across T_c and coarsening under rollout.

Goal & Strategy

Goal: model the *dynamics* of systems whose governing rules are *generally unknown*.

Strategy: establish the method on a system where the ground truth *is* known — the 2D-Ising model, where a learned model can be scored against the *exact* transition rates.

The 2D Ising model

Spins $s_i = \pm 1$ on a lattice with energy (Hamiltonian) $\mathcal{H} = -J \sum_{\langle ij \rangle} s_i s_j$ (we set J=1), where $\langle ij \rangle$ runs over *nearest-neighbour* pairs. It is the **canonical model of phase transitions and coarsening**[3]: below a critical temperature $T_c \approx 2.27$ it orders into growing domains, above T_c it is disordered.

System & data

2D Ising, $L \times L$ lattice (L=16), periodic; temperature T. **Rejection-free KMC**[1]: flipping site i changes the energy by ΔE_i and has rate $w_i = \min(1, e^{-\Delta E_i / T})$[2]; pick site i with probability w_i/R (total rate $R = \sum_i w_i$) and advance the clock by waiting time $\Delta t \sim \mathrm{Exp}(R)$.

Tokenization. Vocabulary: a T-bin per temperature, two spin tokens ($\uparrow, \downarrow$), N=L^2 position tokens, and log-binned Δt tokens. A window of W events tokenizes as:

$$[\,T\,]\ \underbrace{[\,\text{initial config tokens}\,]}_{\times 2}\ \underbrace{[\,T\,][\,\text{pos}\,][\,\Delta t\,]}_{\times W}$$

Example (2×2):

$$[\,T{=}1.5\,]\ \underbrace{[\text{pos}_0][\uparrow][\text{pos}_1][\downarrow][\text{pos}_2][\downarrow][\text{pos}_3][\uparrow]}_{\text{config, written twice}}\ [\,T{=}1.5\,][\text{pos}_3][\Delta t{=}0.4]\ [\,T{=}1.5\,][\text{pos}_1][\Delta t{=}1.2]$$

Data. 14 discrete T: 6 cold $[1.5, 2.0]$ + 8 hot $[2.8, 3.5]$; **critical window** $(2.0, 2.8)$ **around** T_c **held out**. Trajectories equilibrated to steady magnetization $|m| = |\frac{1}{N}\sum_i s_i|$ before recording.

Model & training

- **Architecture:** an **off-the-shelf decoder-only transformer**[4] (GPT-style, with rotary position embeddings, RoPE[5]). d_{model}=256, 6 layers, 8 heads, feed-forward width 1024; vocab ≈339; context 1280; **~5 M parameters**.
- **Objective:** next-token cross-entropy on the (pos, Δt) event tokens only.
- **Data:** 70,000 KMC trajectories (14 temperatures × 5,000), 500 events each ⇒ ~0.7 M training windows.
- **Training:** ~18 epochs, batch 256, AdamW, lr 10^{-3} cosine, **fp32** — ~6.7 h on a single **TPU v6e-4** (4 chips).

Built on Marin

Marin (marin.community) is an open lab for building foundation models openly. MarinSpin uses its model & training stack.

Δt tokens & autoregressive rollout

Waiting-time tokens. The continuous $\Delta t \sim \mathrm{Exp}(R)$ is quantized into log-spaced bins (+ under/overflow); MarinSpin predicts a *category* over Δt-bins.

Rollout. Autoregressive, one event at a time: from the current config MarinSpin emits $p(\text{flip site}) \rightarrow$ sample i, then $p(\Delta t) \rightarrow$ sample a bin and draw Δt log-uniformly within it; flip spin i, advance the clock by Δt, append, repeat.

The oracle floor

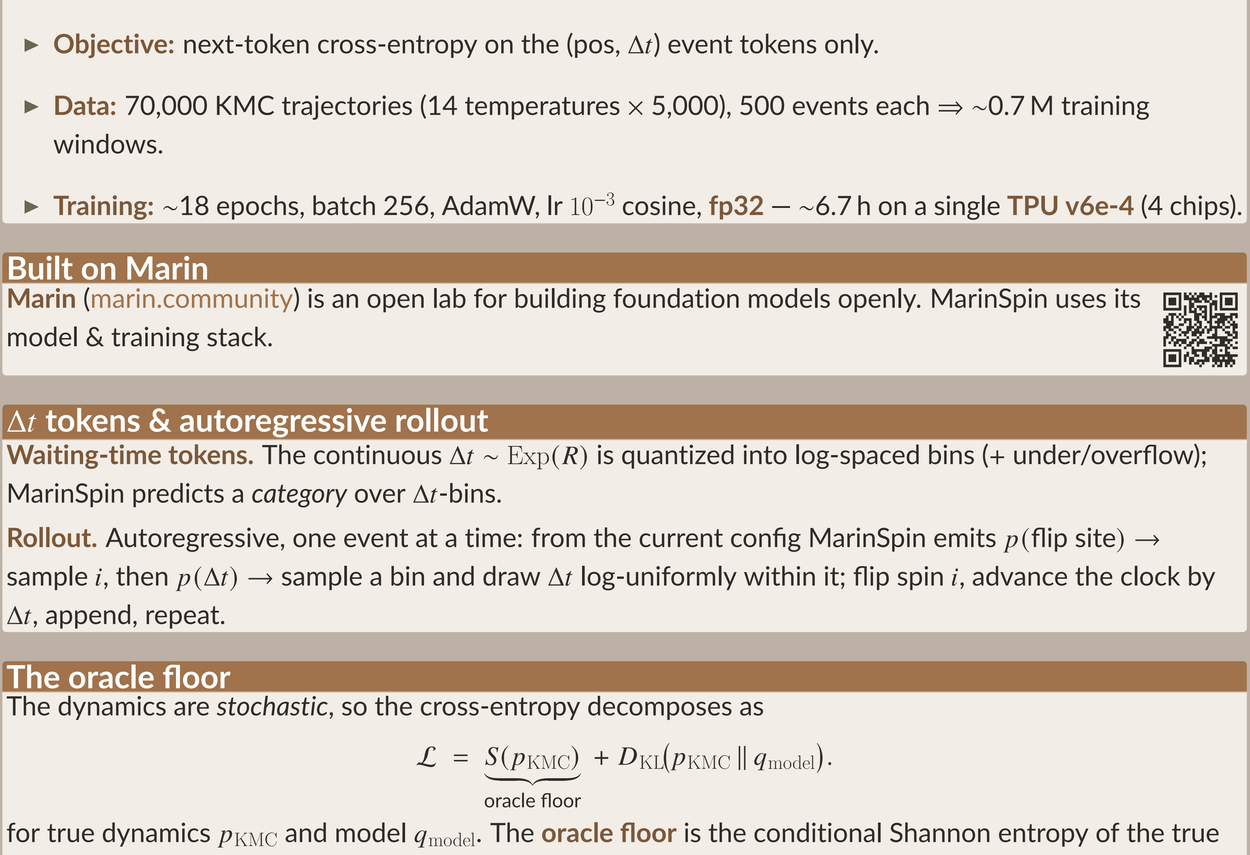
The dynamics are *stochastic*, so the cross-entropy decomposes as

$$\mathcal{L} = \underbrace{S(p_{\text{KMC}})}_{\text{oracle floor}} + D_{\text{KL}}(p_{\text{KMC}} \,\|\, q_{\text{model}}).$$

for true dynamics p_{KMC} and model q_{model}. The **oracle floor** is the conditional Shannon entropy of the true dynamics; the **Kullback–Leibler divergence** D_{KL} is the residual loss:

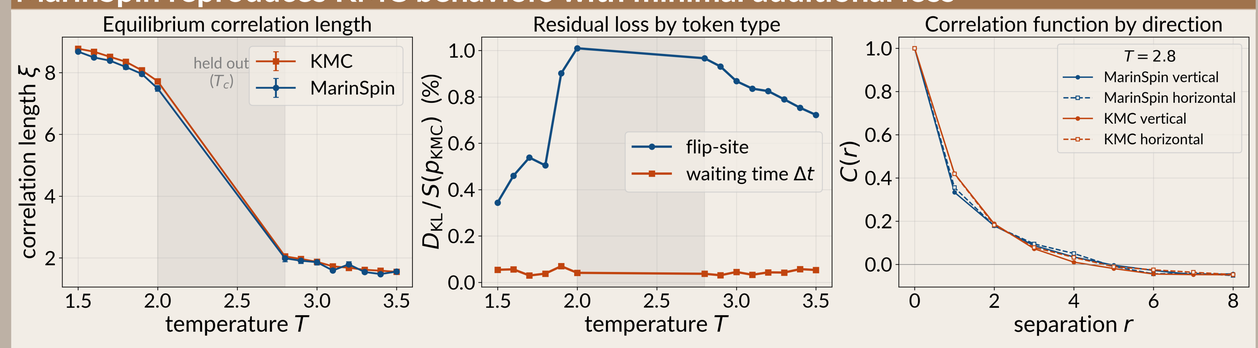
	T=1.5 (cold)	T=3.5 (hot)
oracle floor S (nats)	3.435	4.274
MarinSpin loss $\mathcal{L}$	3.43	4.30
residual D_{KL}	0.012	0.038

Out of distribution (OOD) quench reproduces coarsening dynamics



MarinSpin (top) vs. KMC (bottom) from a checkerboard quench (T=1.5, sampling temp 0.9); panels matched by *event count*. Domains grow and ξ rises together with KMC.

MarinSpin reproduces KMC behaviors with minimal additional loss



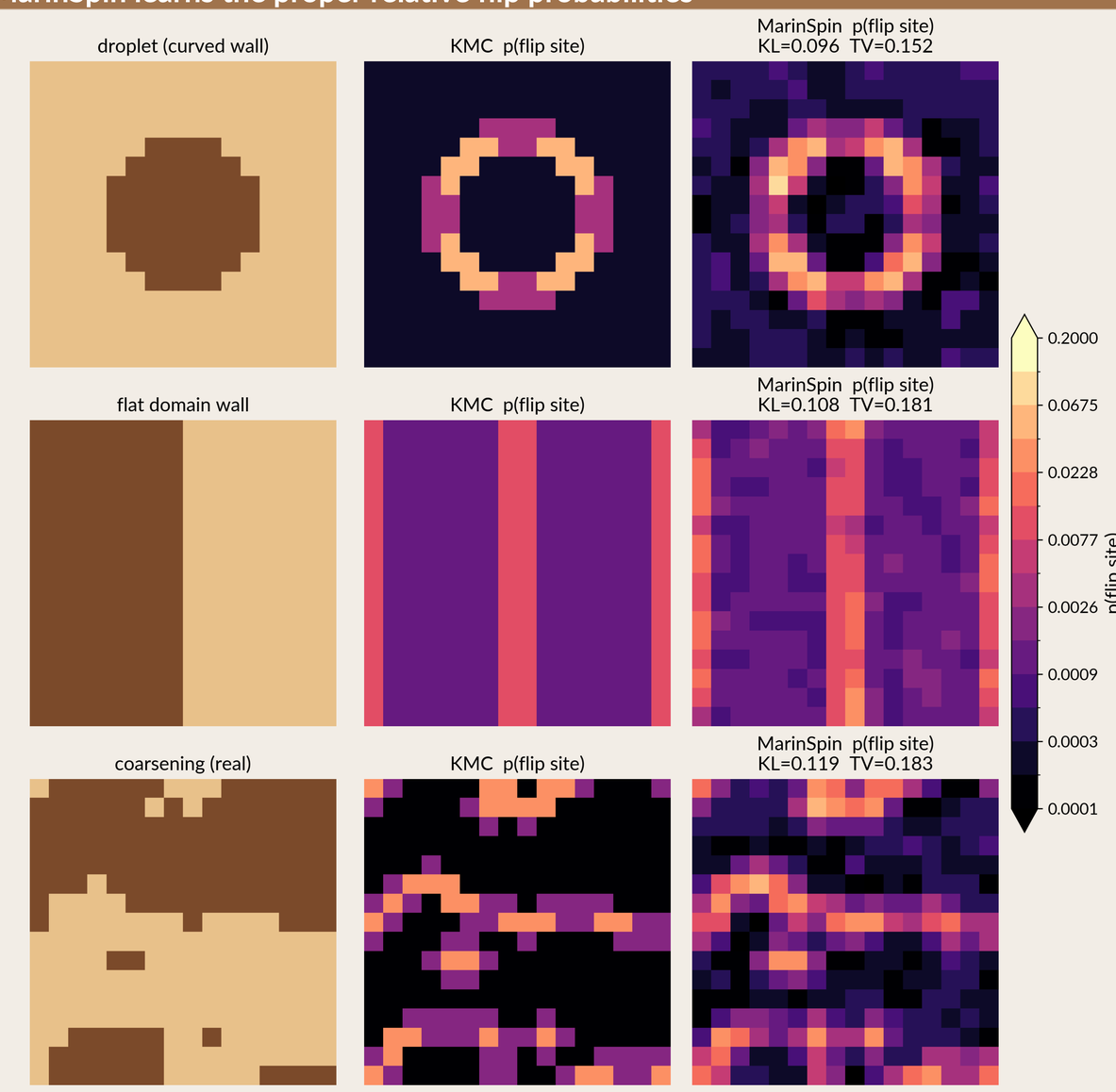
The two-point correlation $C(r)$=$\langle s_0 s_r \rangle$ measures how aligned two spins a distance r apart are; the **correlation length** ξ sets its decay, $C(r) \sim e^{-r/\xi}$.

Left: equilibrium correlation length ξ vs. T.

Middle: residual $D_{\text{KL}}/S(p_{\text{KMC}})$ (%).

Right: the full $C(r)$ by direction (vertical vs. horizontal).

MarinSpin learns the proper relative flip probabilities



Per row: **spin config** → exact KMC p(flip) → model p(flip), T=1.5.
p(flip site i) is the probability that site i is the next to flip (KMC: w_i/R).

Data, training & prompt lessons

- **Coverage:** more *real* equilibrated data cuts per-step KL (–26% cold, –37% hot); the hot phase is coverage-limited.
- **Augmentation:** a limited trial of the 8 square symmetries (rotations/reflections, D_4) blurred the OOD flip probabilities, no clear benefit.
- **Prompt redundancy:** raster written *twice*; dropping the copy **seemingly diverged the loss** (single trial).
- **Over-specialization:** longer training helps in-distribution KL but *degrades* OOD fidelity.
- **Precision:** cold rate ratios span ~200× — **fp32** ≫ **bf16** (32-bit floats capture more nuance in the parameter numbers than the coarser 16-bit bf16).

Next steps

- **Continuous-T encoding:** T is currently a *discrete* token, so MarinSpin can only simulate the exact trained temperatures. A continuous/embedded T is the prerequisite for extending to the critical region.
- **v2 tokenization — neighbor lists:** feed the lattice connectivity (edge / neighbor list) explicitly.
- Larger lattices; **conserved-order-parameter** dynamics.

References

[1] Bortz *et al.*, *J. Comput. Phys.* 17, 10 (1975).
[2] Metropolis *et al.*, *J. Chem. Phys.* 21, 1087 (1953).
[3] Bray, *Adv. Phys.* 43, 357 (1994).
[4] Vaswani *et al.*, *NeurIPS* (2017).
[5] Su *et al.*, *arXiv* 2104.09864 (2021).

Acknowledgements

We thank **Antonia S. J. S. Mey** and **Matteo T. Degiacomi** (University of Edinburgh) for helpful discussions during the early stages of this work, and the **Google TPU Research Cloud (TRC)** for compute.